Spain Electricity Demand Forecasting

Can We Beat the TSO?

Advanced Time Series Forecasting – Final Assignment

Iker Urdiroz Melchor Lafuente Iker Jáuregui

April 13, 2026

1 Introduction

Accurate electricity demand forecasting is a critical task for Transmission System Operators (TSOs). Every megawatt-hour of error has a direct financial and environmental cost, and sustained forecasting failures can cascade into system-wide frequency imbalances. The Spanish TSO publishes its own official day-ahead demand forecast every day, produced by experienced engineers using proprietary models and decades of operational knowledge.

The objective of this work is to develop a forecasting system that predicts hourly national electricity demand across the Spanish peninsula and, ultimately, to try to beat the official TSO forecast. We approach this challenge by following an incremental roadmap: starting from simple statistical baselines, moving through classical ML models, and finally exploring deep learning architectures. Along the way, we enrich the feature set with external covariates sourced from different domains, and we critically analyse when and why our models outperform or underperform the TSO benchmark.

2 Datasets

2.1 Energy and Weather (Kaggle)

The core dataset is the publicly available *Hourly Energy Demand, Generation and Weather in Spain* from Kaggle, covering the period 2015–2018 at hourly resolution. It contains two linked files:

- **Energy dataset:** includes the target variable `total load actual` (MWh), the TSO forecast (`total load forecast`), generation mix by technology, and day-ahead prices.
- **Weather dataset:** hourly meteorological observations (temperature, humidity, wind speed, pressure, cloud cover, weather description) for five major Spanish cities: Madrid, Barcelona, Valencia, Seville, and Bilbao.

2.2 External Covariates

Beyond the Kaggle dataset, we sourced and processed four additional groups of exogenous covariates:

- **Macroeconomic (Macro):** Brent crude oil price (converted to EUR/barrel via the EUR/USD exchange rate), TTF natural gas price (EUR/MWh), and the EUR/USD exchange rate itself. Energy commodity prices are a fundamental driver of electricity generation costs and, indirectly, of demand-side behaviour.
- **Sports:** Binary/Gaussian features encoding major Spanish football events (El Clásico, Madrid Derby, Catalan Derby, and Spanish national team matches in Euro 2016 and World Cup 2018). High-viewership matches are a documented source of short-term demand anomalies.
- **Calendar:** Spanish national holiday flags, weekend indicators, cyclical encodings of the day of the week and month of the year (sine/cosine transforms), and a normalized year feature to capture secular trends.
- **Holidays:** National holidays derived from the `holidays` Python library for Spain.

3 Methodology

3.1 Data Cleaning and Preparation

3.1.1 Energy Dataset

Columns composed entirely of zeros or NaN values (e.g. `generation fossil coal-derived gas`, `generation wind offshore`) were removed as they contain no usable information. We also dropped forecasted variables and price columns to prevent temporal leakage.

Regarding outliers, most generation columns contained a small number of suspicious zero values concentrated at three specific timestamps (`2017-11-12 20:00`, `2017-11-14 11:00`, `2017-11-14 18:00`), which coincided across almost all variables. This pattern strongly suggests data corruption rather than legitimate readings, so those values were set to NaN.

For the target variable (`total load actual`), 36 missing values were found, mostly consisting of punctual hourly gaps with two larger gaps of a few hours. We compared linear, quadratic, and cubic interpolation methods and selected **quadratic interpolation** (Figure 1), as it best replicated the curvy patterns characteristic of electricity demand. For the remaining generation covariates, which had similar small gaps at shared timestamps, linear interpolation was applied; the noise introduced by this choice is expected to be negligible.

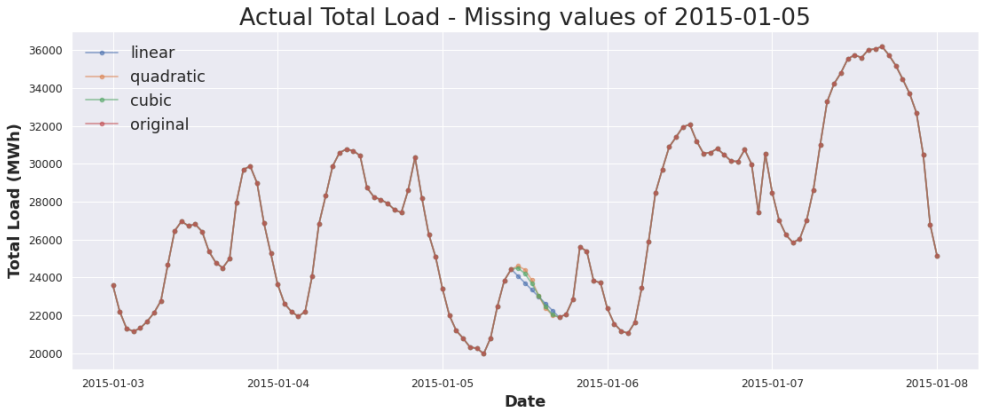

Figure 1: Tested interpolation methods.

3.1.2 Exogenous Covariates

Weather data was cleaned by converting temperature from Kelvin to Celsius and wind speed from m/s to km/h. Wind direction was decomposed into Cartesian components ($\text{wind\_x} = \cos\theta$, $\text{wind\_y} = \sin\theta$) to avoid the artificial discontinuity at the 360°/0° boundary. Physical-limit outlier detection was applied to pressure (870–1085 hPa), humidity ($< 10\%$), and wind speed (> 180 km/h); values outside these bounds were treated as sensor errors, set to NaN, and imputed via linear interpolation. The categorical weather description (30+ labels) was simplified into three levels (*good*, *mid*, *bad*) and one-hot encoded.

For macroeconomic variables, weekend gaps were filled via forward-fill (assuming the last known price remains valid), and the series were expanded to hourly frequency.

3.1.3 Data Leakage Prevention

Two critical decisions were taken to prevent temporal leakage:

1. **Energy generation covariates** were shifted by 24 hours, so that at prediction time the model only sees generation data from the previous day.

2. **All financial variables** (Brent, TTF, EUR/USD) were shifted by one calendar day, ensuring that only closing prices available before the prediction window are used.

Additionally, the actual weather observations used as covariates represent an *oracle assumption* (perfect weather forecast). In a real deployment, these would be replaced by day-ahead weather forecast APIs, which would introduce additional forecast error.

3.1.4 Merge

After cleaning and leakage prevention, all dataframes (target, energy covariates, and exogenous covariates) were joined on their UTC hourly index, resulting in a final dataset of 35,040 hourly observations and 90 columns.

3.2 Exploratory Data Analysis

3.2.1 Correlation Study

We analysed the correlation between the target variable and all covariates both globally (Pearson over the full dataset) and through quarterly rolling correlation windows (Figure 2).

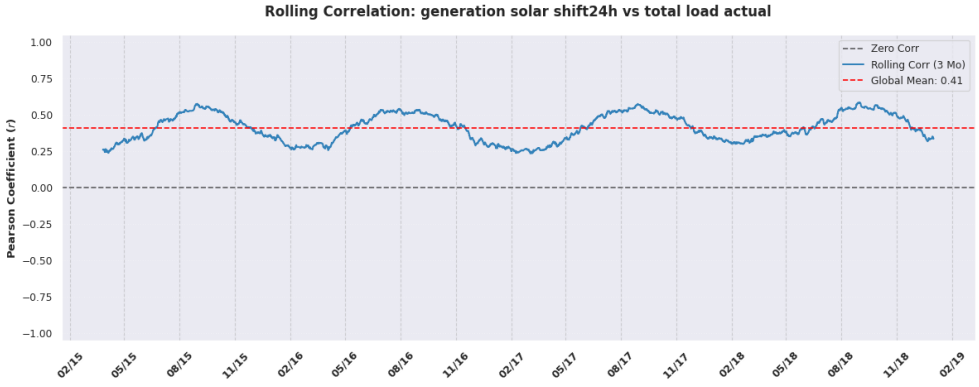

Figure 2: Rolling correlation window applied to `generation solar shift24h`.

The highest absolute correlations were mainly associated with climatic and meteorological covariates. Temperature showed a stronger relationship during warmer periods, while humidity became more relevant during colder seasons. This complementary behaviour suggests that different weather variables capture different demand drivers across the year.

Energy-related covariates such as gas and pumped hydro generation exhibited moderate correlation, coherent with their role as flexible or backup sources whose production adjusts in response to demand. Economic covariates exhibited both positive and negative correlation peaks without clear temporal patterns, requiring further analysis to assess whether they reflect meaningful causal relationships or mainly introduce noise. Calendar variables showed intuitive patterns, with weekends associated with lower demand. Sports events and holidays did not present strong overall correlations, although they may still be relevant at a micro scale.

3.2.2 Trend and Seasonality

The series exhibits a roughly **horizontal trend** with no evident long-term growth or decline. We identified a clear **quarterly cyclic pattern** (Figure 3) through a quarterly moving average, suggesting macro-level demand cycles possibly driven by seasonal economic activity.

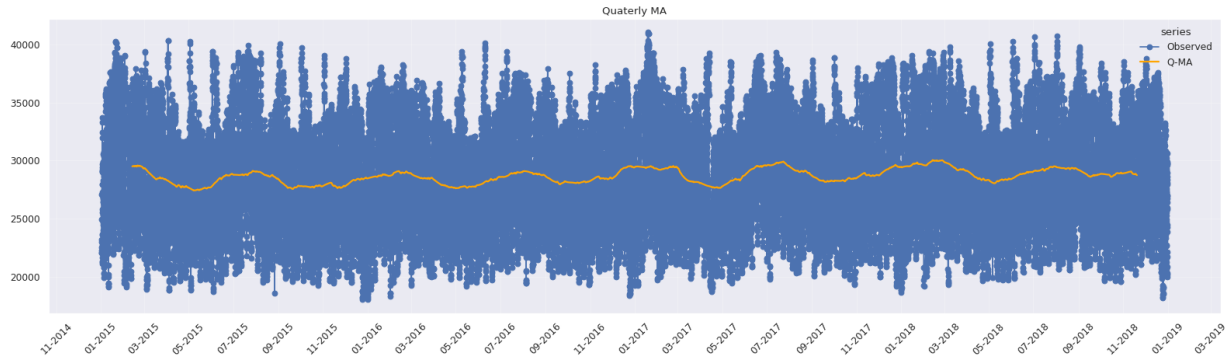

Figure 3: Quarterly cyclic pattern.

At finer resolution, the series shows strong **daily seasonality**: demand rises during daytime hours and falls at night, with consistent descents during weekends (Figure 4). Variance appears approximately constant, supporting the use of an additive decomposition model rather than a multiplicative one.

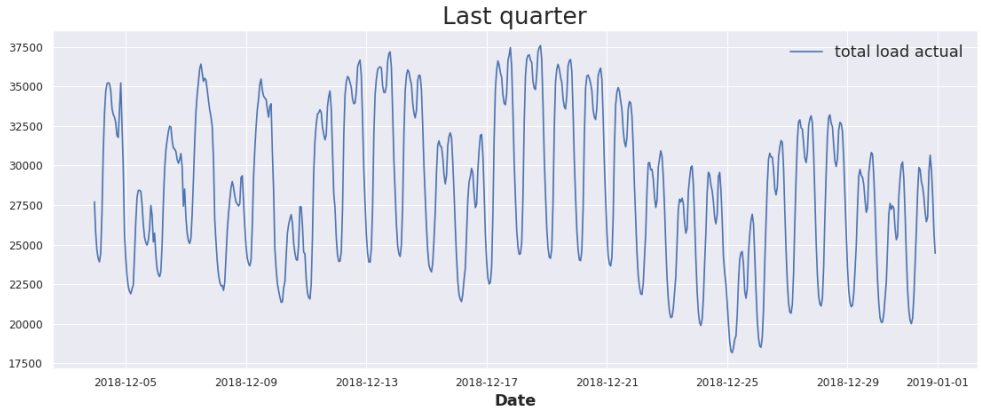

Figure 4: Daily seasonality pattern.

3.3 Train / Validation / Test Split

For time series problems, stratified splitting is not applicable. Instead, we developed a quantitative methodology to select the best split by computing two metrics across candidate configurations:

- **ACF similarity:** measures how similar the autocorrelation structures of validation and test sets are.
- **Naive mean error ratio:** compares the naive MAE on both splits to ensure they represent similarly difficult forecasting problems.

Testing four candidate split sizes (30, 91, 182, and 365 days for each of validation and test), the **365-day** option achieved the highest ACF similarity (0.66) with a competitive naive ratio score (0.98). We therefore used:

- **Train:** 2015-01-01 to 2016-12-31 (730 days)
- **Validation:** 2017-01-01 to 2017-12-31 (365 days)
- **Test:** 2018-01-01 to 2018-12-31 (365 days)

3.4 Evaluation Metrics

Three complementary metrics are used throughout this work:

- **RMSE** and **MAE** are expressed in the same units as the target variable (MWh), which makes them directly interpretable in terms of the physical magnitude of the forecasting error.
- **MAPE** is a dimensionless metric that expresses the error as a percentage of the actual value. This allows us to evaluate model performance on a relative scale, facilitating comparison across different demand regimes and against the TSO benchmark regardless of the absolute load level.

3.5 TSO Evaluation as Reference

Before training any model, we evaluated the TSO forecast on the test set to establish the benchmark:

Metric	TSO Forecast
RMSE	389.32
MAE	269.85
MAPE (%)	0.93

The TSO forecast sets a remarkably high bar, and surpassing it will be a considerable challenge.

3.6 Model Training and Evaluation Strategy

3.6.1 Forecast Horizon

All models are configured with a **24-hour forecast horizon**. This choice is not arbitrary: the TSO produces a day-ahead forecast covering the next 24 hours, so adopting the same horizon ensures that our models are evaluated under equal conditions and that any comparison against the TSO benchmark is fair and operationally meaningful.

3.6.2 Roadmap

We followed an incremental approach:

1. **Statistical models** (SARIMA, SARIMAX) to establish a solid baseline.
2. **ML models** (Prophet, XGBoost) to attempt to solve the problem with more expressive approaches.
3. **DL models** (TCN, N-HiTS) to explore whether deep architectures with large receptive fields could improve further.

3.6.3 Training Protocol

For all models, we followed the standard practice of:

1. Training on the **train set** and searching for the best hyperparameters using the **validation set**.
2. Refitting the model on **train + validation** and evaluating on the **test set**.

3.7 Best Model Selection and Analysis

The best performing model was selected based on test set metrics and subjected to a detailed comparison against the TSO forecast at multiple temporal scales (daily RMSE, hourly MAPE, and day-of-week MAPE).

4 Experiments

4.1 SARIMA

Since the series was already stationary (confirmed by both ADF and KPSS tests), we applied `auto_arima` with $d = 0$ and $D = 1$ (seasonal differencing with $m = 24$ for daily seasonality). Due to memory constraints, the training window had to be limited to 28 days. The best model selected was SARIMA$(1, 0, 2)(0, 1, 1)_{24}$. All coefficients were statistically significant, but the Jarque-Bera test rejected normality of residuals, indicating that the model did not fully capture the series' dynamics.

After refitting on validation data:

	RMSE	MAE	MAPE (%)
SARIMA	5005.39	4135.67	13.63

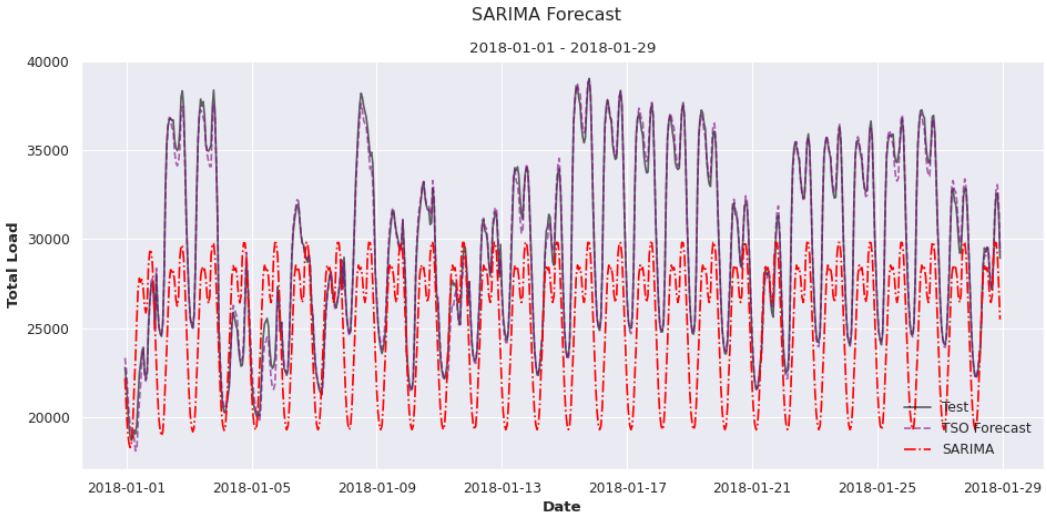

Figure 5: SARIMA forecast on test set.

As expected, the error is considerably higher than the TSO benchmark. Nevertheless, it provides a well-defined baseline against which to measure subsequent improvements.

4.2 SARIMAX

We tested different exogenous covariate blocks (calendar, weather, generation, macro, events) using the same SARIMA parameters. Calendar covariates produced the best validation RMSE (3545 vs. 3729 without covariates), while weather, generation, and macro covariates seemed to confuse the model.

After refitting with calendar covariates:

	RMSE	MAE	MAPE (%)
SARIMAX (calendar)	8052.27	7317.60	24.63

Although calendar covariates improved validation results, the performance decayed on the test set after refitting. We suspect this is due to overfitting over the calendar covariates, caused by the small training window of only 28 days.

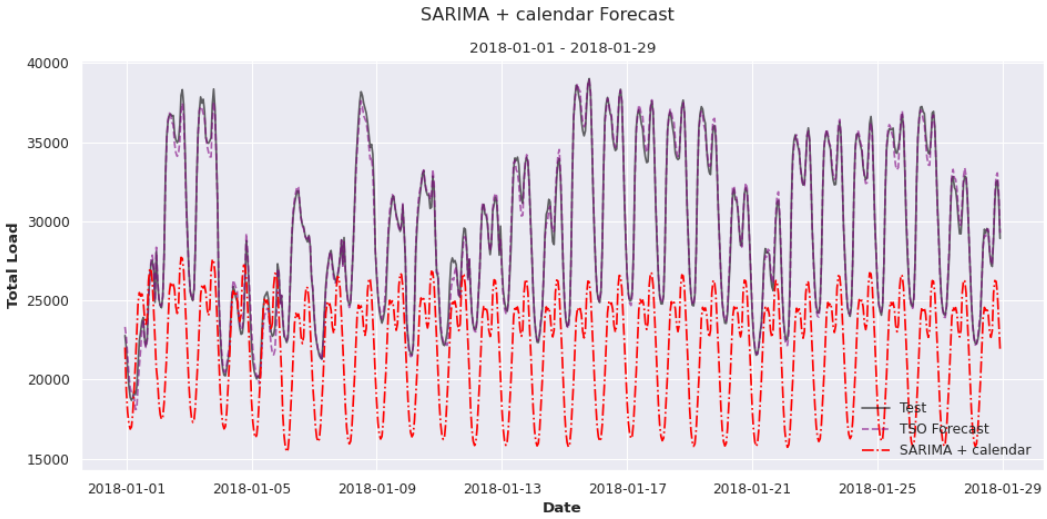

Figure 6: SARIMAX forecast on test set.

4.3 Prophet

We first performed a grid search over Prophet's hyperparameters (`changepoint_prior_scale`, `seasonality_prior_scale`, `seasonality_mode`). Additive models consistently outperformed multiplicative ones, which makes sense given the constant variance of the series. The best configuration was: `changepoint_prior_scale`= 0.01, `seasonality_prior_scale`= 10.0, additive mode.

For covariate selection, we tested different blocks pivoting around the best-performing sets. Energy generation combined with calendar features yielded the best validation RMSE (2720).

After refitting on train + validation:

	RMSE	MAE	MAPE (%)
Prophet (gen. + cal.)	2645.50	1973.55	6.96

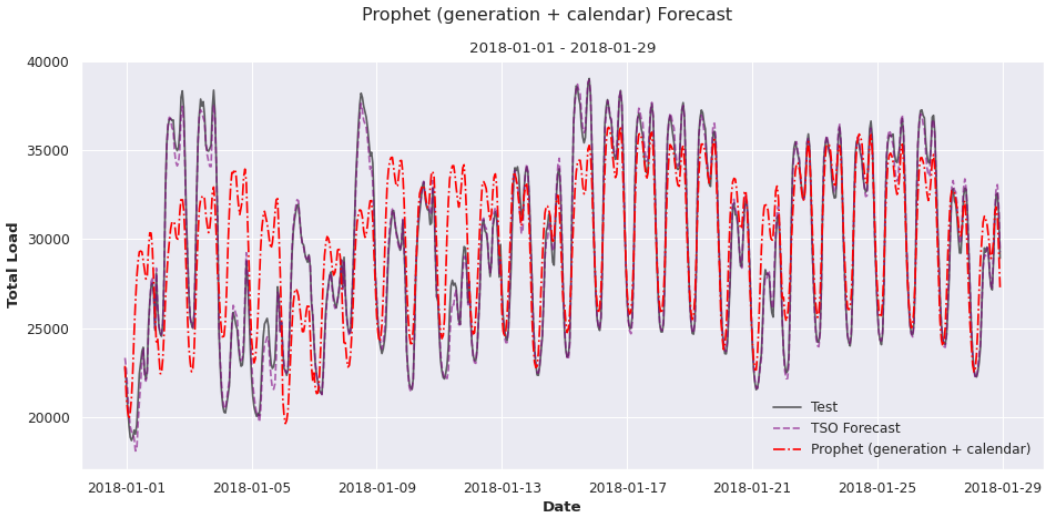

Figure 7: Prophet forecast on test set.

Although still far from the TSO benchmark, the results represent a substantial improvement over SARIMA and SARIMAX. Training on the full train and validation data prevented the overfitting observed in previous models. Nevertheless, Prophet was still unable to leverage climatic covariates effectively.

4.4 XGBoost

To insert temporal context into XGBoost, we engineered lag features (1h, 2h, 3h, 24h, 48h, 168h) and rolling window statistics (mean, std, min, max over 24h and 168h windows). We used `ForecasterRecursive` from `skforecast` with a 24h forecast horizon.

Three progressive experiments were conducted:

1. **Base:** Lag/rolling features of the target only → $\text{RMSE}_{\text{val}} = 1382.71$
2. **Duplicated lags:** After SHAP analysis revealed the model heavily relied on `lag_1`, we duplicated lag features in the `ForecasterRecursive` configuration → $\text{RMSE}_{\text{val}} = 1243.45$
3. **Generation lags:** Added lag/rolling features for energy generation covariates → $\text{RMSE}_{\text{val}} = 1184.09$

SHAP analysis revealed that the model mainly relies on lagged features of the target, solar generation, and hydro pumped generation. Solar generation is related to weather, and hydro pumped generation is a backup system—so the model is finally taking advantage of different data sources.

Best hyperparameters: `n_estimators`= 1200, `learning_rate`= 0.01, `max_depth`= 7, `colsample_bytree`= 0.25.

After refitting on train + validation:

	RMSE	MAE	MAPE (%)
XGBoost (lag + roll + exog)	1095.17	789.94	2.77

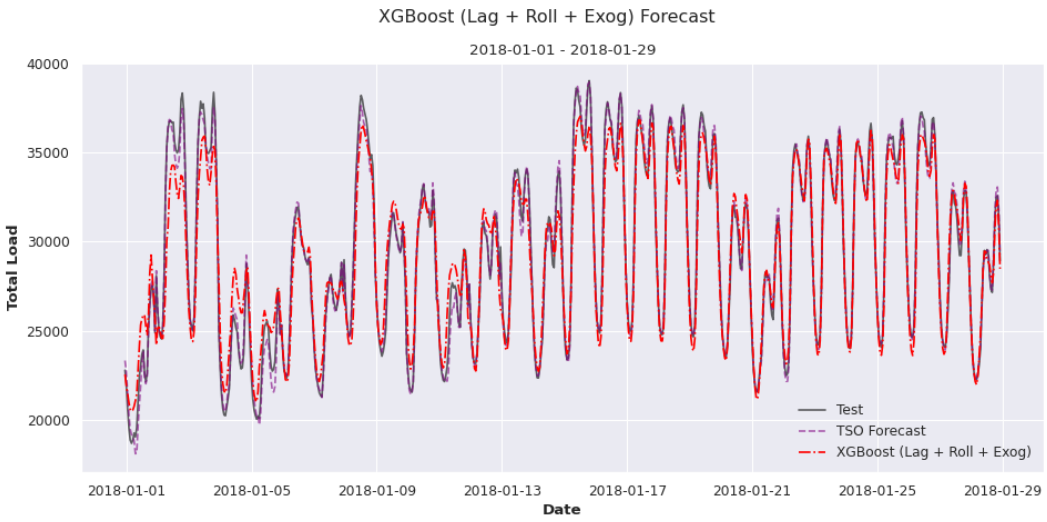

Figure 8: XGBoost forecast on test set.

We believe the improvement is mainly due to the context window carried through lagged variables.

4.5 TCN (Temporal Convolutional Network)

We hypothesised that a TCN could generate the aggregation functions it needs (instead of fixed mean, std, etc.) while covering a large receptive field. We started with a configuration matching XGBoost's context window (RF ≈ 243h with kernel= 3, dilation base= 3, 5 layers).

The model suffered from severe overfitting across multiple configurations. We tried reducing filters (64→16→8), increasing dropout (0.1→0.5), reducing learning rate, removing covariates, and shrinking the input chunk. The best training behaviour (without overfitting) came from a minimal architecture without covariates, but it converged to something like a Naive Mean forecaster (Figure 9).

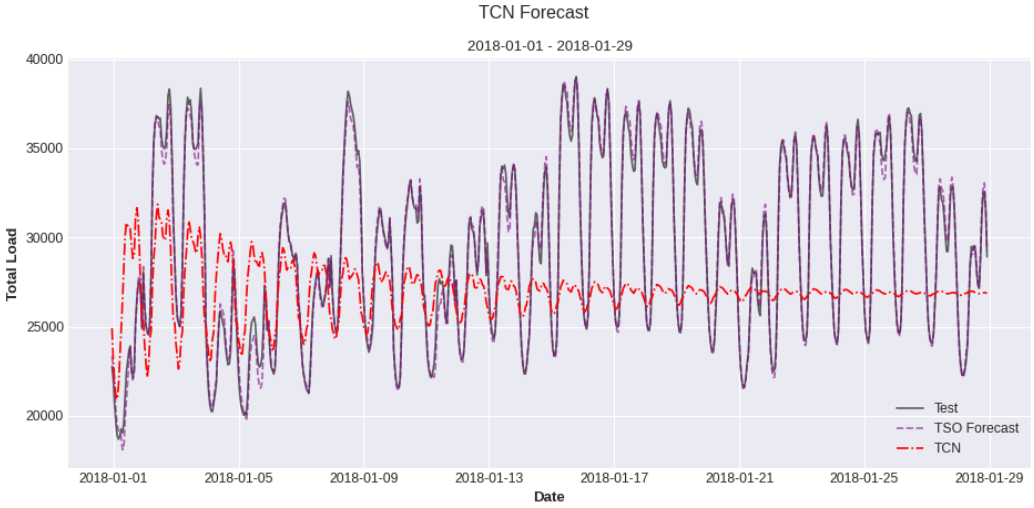

Figure 9: TCN forecast that resembles a Naive Mean forecaster.

We finally trained an overfitting-prone configuration on train + validation (without early stopping) to leverage more data:

	RMSE	MAE	MAPE (%)
TCN (with exog)	4067.13	3183.61	10.77

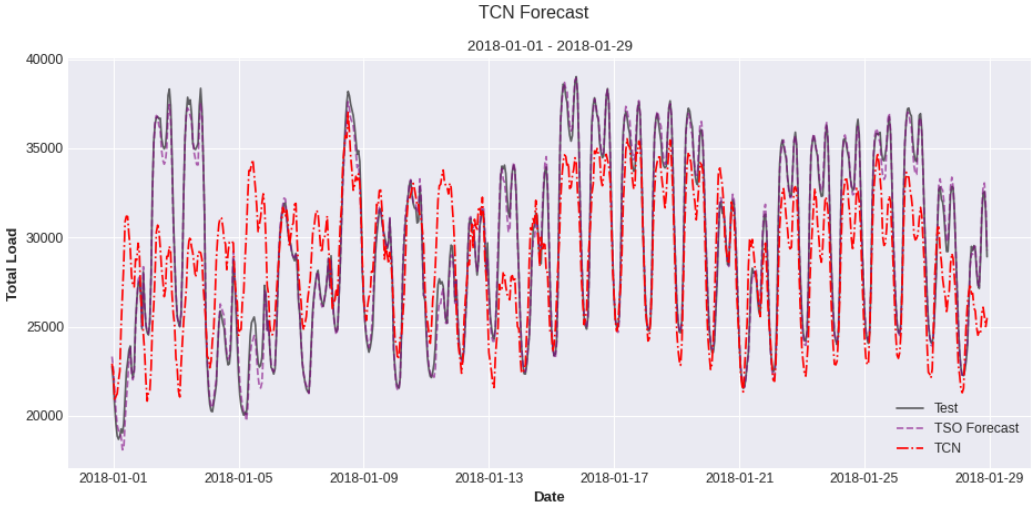

Figure 10: Final TCN forecast on test set.

We believe the exogenous covariates do benefit this architecture, but the high dimensionality makes it considerably data hungry, and 2–3 years of hourly data proves insufficient to adequately train the network.

4.6 N-HiTS

N-HiTS was selected as the final model due to its hierarchical pooling system, which should provide more regularisation by design than TCN. We configured three pooling scales (weekly, daily, hourly) with progressively simplified architectures.

Despite extensive regularisation (reducing blocks, layers, widths, increasing dropout, switching to Huber loss), the model consistently overfitted. The final configuration used two stacks (daily and

hourly pooling), one block, one layer, 32 hidden units, dropout 0.3, and Huber loss:

	RMSE	MAE	MAPE (%)
N-HiTS (with exog)	3271.60	2651.46	9.50

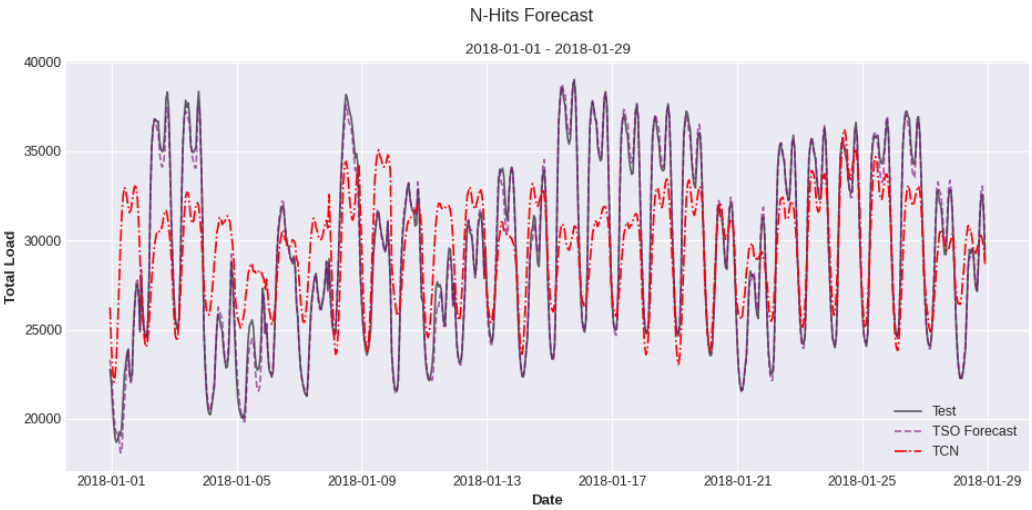

Figure 11: N-Hits forecast on test set.

The N-HiTS behaved similarly to the TCN, probably suffering from the same data scarcity problem.

4.7 Summary of Results

Table 1: Test set performance of all models compared to the TSO forecast.

Model	**RMSE**	**MAE**	**MAPE (%)**
TSO Forecast	**389.32**	**269.85**	**0.93**
SARIMA	5005.39	4135.67	13.63
SARIMAX (calendar)	8052.27	7317.60	24.63
Prophet (gen. + calendar)	2645.50	1973.55	6.96
XGBoost (lag + roll + exog)	**1095.17**	**789.94**	**2.77**
TCN (with exog)	4067.13	3183.61	10.77
N-HiTS (with exog)	3271.60	2651.46	9.50

5 Analysis of Results

XGBoost was the best performing model with a MAPE of 2.77%, although the TSO forecast still outperforms it significantly (MAPE of 0.93%). We conducted a detailed comparison between the two.

Daily RMSE analysis. (Figure 12) XGBoost beats the TSO on only 12 out of 366 days (3.3%). Most of those days fall in August and December, which we thought it could be related to heat/cold waves. We analysed the temperature during the August dates and found they were slightly colder than the historical median, but not cold enough to clearly attribute the TSO error to a cold wave. Further research would be necessary.

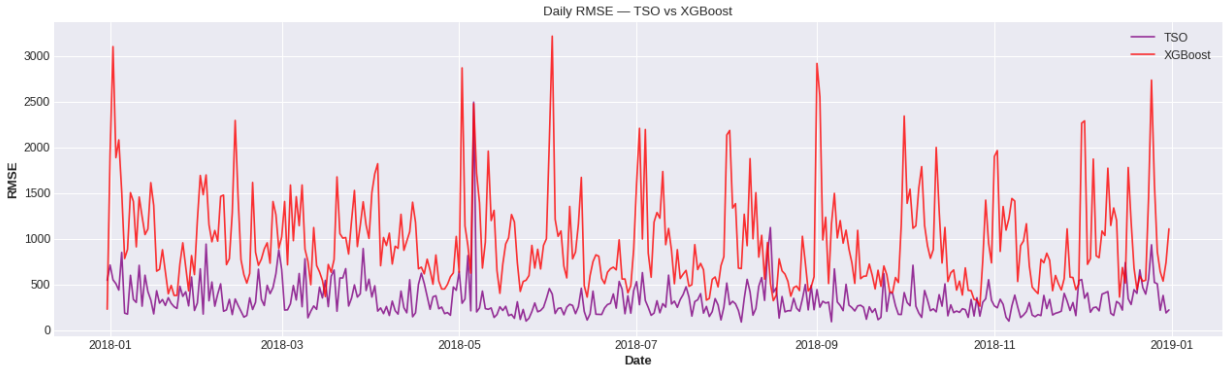

Figure 12: Daily RMSE analysis between TSO and XGBoost forecasts.

Hourly MAPE analysis. (Figure 13) Even if our model always commits bigger errors than the TSO, its relative performance is better where the TSO performs worse—specifically during mid-day and afternoon hours.

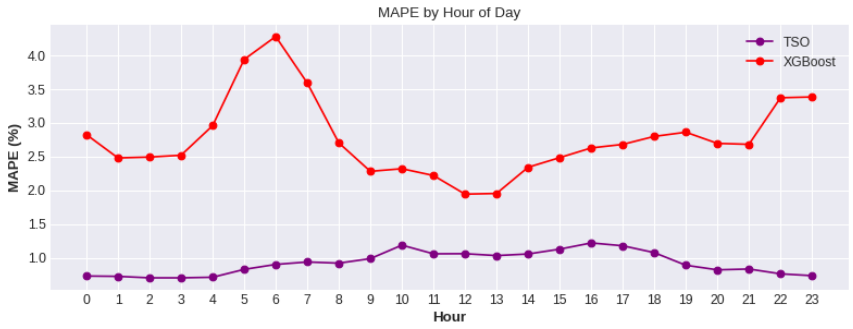

Figure 13: Hourly MAPE analysis between TSO and XGBoost forecasts.

Day-of-week MAPE analysis. (Figure 14) Similarly, XGBoost shows relatively better performance on Thursdays, which coincides with the day where the TSO shows its highest error.

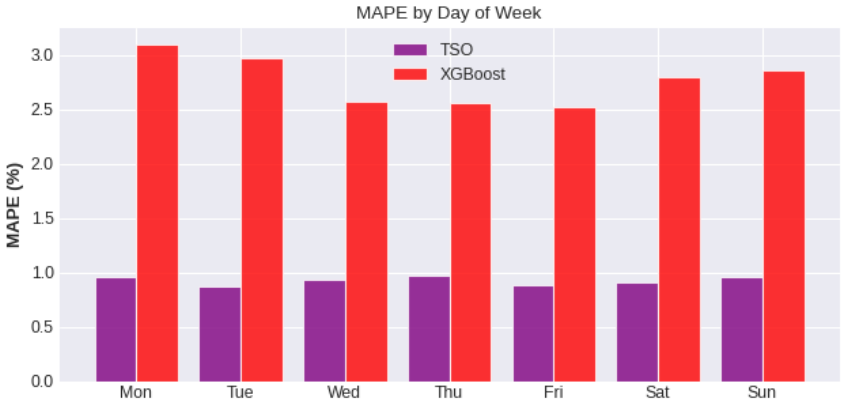

Figure 14: Day-of-week MAPE analysis between TSO and XGBoost forecasts.

Sports event analysis. (Figure 15) We checked whether the days where XGBoost beats the TSO correspond to football events. Only one day (2018-05-06) coincided with a La Liga match, but the observed demand dip did not align with the match time. Both models failed at that point, with ours only marginally better.

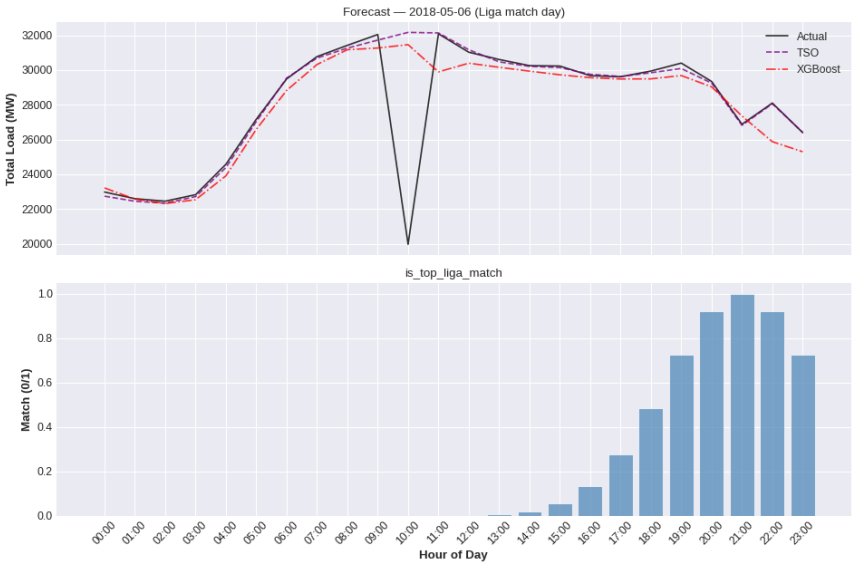

Figure 15: Hourly MAPE analysis between TSO and XGBoost forecasts.

6 Conclusions

During this work we tested several different models from different paradigms. Statistical models like SARIMA and SARIMAX couldn't capture the complexity of the series, and data-hungry models like TCN and N-HiTS overfitted due to the lack of data. The best results were obtained with something in-between: ML models. Prophet improved over SARIMA and SARIMAX but still committed quite big errors. XGBoost resulted the best model by far, with a MAPE of 2.77%, although the TSO forecast outperforms it (MAPE of 0.93%).

The performance comparison and analysis between the TSO and our XGBoost revealed that the boosting model beat the TSO on some punctual days. Unfortunately, due to our lack of domain expertise, we couldn't find a significant reason for that behaviour. However, we found an interesting pattern at the hourly and day-of-week scale: **XGBoost performs better where the TSO forecast performs worse**. This may suggest that an XGBoost fitted on the TSO residuals, or fitted with the TSO forecast (lagged by 24h at least) as an input feature, could reach better results. This would be a promising direction for future research.